Complete OpenROAD Installation & Troubleshooting Guide (WSL Ubuntu)

Overview

This document is a complete practical guide for installing and building the OpenROAD ASIC flow environment on Ubuntu/WSL, based on a real installation journey including all major errors, dependency issues, Docker problems, submodule issues, and build failures encountered during setup.

The goal of this guide is not just to install OpenROAD, but to understand:

- Why errors happen
- What each dependency does
- How to debug installation failures
- How to avoid rebuilding everything repeatedly
- How to successfully reach RTL-to-GDSII flow execution

This guide is written specifically for:

- Ubuntu users
- WSL2 users on Windows
- Beginners entering ASIC/VLSI physical design
- Students using OpenROAD for RTL-to-GDSII projects

System Used

Recommended Environment

Component	Recommendation
OS	Ubuntu 22.04
Environment	WSL2
RAM	Minimum 16 GB recommended
CPU Threads	8+ preferred
Storage	30+ GB free

Important Understanding Before Installation

OpenROAD is NOT a single tool.

It is a large ecosystem consisting of:

- Yosys
- OpenROAD
- OpenSTA
- TritonRoute
- FastRoute
- KLayout
- Magic
- OR-Tools
- Boost
- SWIG
- TCL
- Python libraries
- CMake infrastructure
- Multiple git submodules

Because of this:

- dependency mismatch errors are common
- build failures are normal
- partial builds are common
- rebuilding from scratch repeatedly wastes time

The biggest challenge is usually NOT Verilog.

It is environment setup.

Initial Installation

Step 1: Install Basic Packages

```
sudo apt update

sudo apt install -y
build-essential
git
cmake
tcl
tcl-dev
```

```
libtcl8.6
libreadline-dev
bison
flex
clang
gcc
g++
python3
python3-pip
python3-dev
curl
wget
pkg-config
zlib1g-dev
libffi-dev
libboost-all-dev
```

Cloning OpenROAD-flow-scripts

```
git clone --recursive https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.git

cd OpenROAD-flow-scripts
```

IMPORTANT:

Always use:

```
git clone --recursive
```

Otherwise submodules will be missing.

First Major Error Encountered

SWIG Version Error

Error

```
Could NOT find SWIG: Found unsuitable version "4.2.0",
but required is at least "4.3"
```

Why This Happens

Ubuntu repositories often provide:

```
SWIG 4.2.0
```

But newer OpenROAD versions require:

```
SWIG >= 4.3
```

The system detects:

```
/usr/bin/swig
```

instead of your newly built version.

Solution

Build SWIG 4.3 Manually

Download

```
wget https://github.com/swig/swig/archive/refs/tags/v4.3.0.tar.gz
```

Extract

```
tar -xzf v4.3.0.tar.gz
cd swig-4.3.0
```

Build

```
./autogen.sh
./configure
make -j$(nproc)
sudo make install
```

Important Verification

After installation:

```
which swig
```

Should show:

```
/usr/local/bin/swig
```

NOT:

```
/usr/bin/swig
```

Then:

```
/usr/local/bin/swig -version
```

Should show:

```
SWIG Version 4.3.0
```

Second Major Problem

CMake Still Detects Old SWIG

Even after installing SWIG 4.3.

Why This Happens

CMake cache remembers previous paths.

It continues using:

```
/usr/bin/swig4.0
```

from older cache data.

Solution

Delete build directory/cache.

```
rm -rf tools/OpenROAD/build
```

Then rerun configuration.

This forces CMake to rediscover dependencies.

Third Major Issue

Missing Dependency Cascade

After SWIG was fixed, multiple dependencies started failing one-by-one.

Examples:

```
Could not find absl
Could not find spdlog
```

```
Could not find Eigen3
Could not find LEMON
Could not find ortools
```

Why This Happens

OpenROAD uses modern CMake package discovery.

Each library requires:

- headers
- shared libraries
- cmake config files

Missing ANY of them causes failure.

Fixes

absl

```
sudo apt install libabsl-dev
```

---

spdlog

```
sudo apt install libspdlog-dev
```

Eigen3

```
sudo apt install libeigen3-dev
```

LEMON

```
sudo apt install liblemon-dev
```

OR-Tools

This dependency is harder.

Sometimes system packages are outdated.

In many cases OpenROAD installs OR-tools internally.

Avoid mixing random OR-tools versions from internet tutorials.

Major Docker Attempt

At one point Docker was used to simplify installation.

Example:

```
docker run -it -v $PWD:/OpenROAD-flow-scripts openroad/flow-ubuntu22.04-builder
bash
```

Docker Problem 1

Missing Yosys

Error

```
YOSYS_EXE is set but either not found or not executable
```

Why This Happens

The Docker image does not automatically contain locally built tools.

The flow expected:

```
tools/install/yosys/bin/yosys
```

which did not exist yet.

Solution

Run:

```
./build_openroad.sh --local
```

inside container.

Docker Problem 2

Dubious Ownership Git Errors

Error

```
fatal: detected dubious ownership in repository
```

Why This Happens

WSL Windows-mounted files appear owned differently inside Docker.

Git blocks operations for security.

Solution

Add safe directories manually.

Example:

```
git config --global --add safe.directory /OpenROAD-flow-scripts
```

Then more subdirectories also needed:

```
git config --global --add safe.directory /OpenROAD-flow-scripts/tools/OpenROAD
```

and:

```
git config --global --add safe.directory /OpenROAD-flow-scripts/tools/OpenROAD/
src/sta
```

and more.

Better Solution Eventually Used

A clean rebuilt local environment was easier than fighting nested Docker ownership repeatedly.

Massive CMake Cache Path Error

Error

```
The current CMakeCache.txt directory is different than the directory where
CMakeCache.txt was created
```

Why This Happens

Build directories were copied/moved between:

```
/home/conolas/
```

and:

```
/OpenROAD-flow-scripts/
```

CMake stores absolute paths.

Moving projects invalidates cache.

Solution

Delete build directories completely.

```
rm -rf tools/OpenROAD/build
```

Then rebuild.

Never move partially built OpenROAD directories.

Boost Version Conflict

Error

```
Could not find boost_iostreams that exactly matches requested version 1.87.0
```

Detected:

```
1.89.0
```

Why This Happens

Multiple Boost versions existed:

- system Boost
- manually installed Boost
- OR-tools bundled Boost

CMake selected conflicting versions.

Important Lesson

Avoid manually installing random Boost versions unless absolutely necessary.

Version conflicts are extremely common.

Major Build Failure During Compilation

Error

```
g++: fatal error: Killed signal terminated program cc1plus
```

Why This Happens

This is usually NOT a compiler bug.

Linux kills the compiler because:

- RAM exhausted
- swap exhausted
- too many parallel threads

OpenROAD compilation is memory intensive.

Solution

Reduce thread count.

Instead of:

```
make -j12
```

Use:

```
make -j4
```

or:

```
make -j2
```

Important Memory Lesson

Higher thread count is NOT always faster.

If RAM is insufficient:

- builds fail
- system freezes
- compiler gets killed

Stable lower thread count is better.

Final Successful Setup Verification

Commands used:

```
which openroad
which yosys
which klayout
```

Outputs:

```
/root/clean_orfs/tools/install/OpenROAD/bin/openroad
/root/clean_orfs/tools/install/yosys/bin/yosys
/usr/bin/klayout
```

This confirmed:

- OpenROAD installed
- Yosys installed
- KLayout accessible

Important Lessons Learned

1. Delete Build Cache Frequently

When changing dependencies:

```
rm -rf build
```

is often necessary.

---

2. Do NOT Mix Too Many Tutorials

Different tutorials use:

- different Ubuntu versions
- different OpenROAD commits
- different dependency versions

Mixing them causes conflicts.

---

3. Docker Is Helpful But Not Magic

Docker simplifies dependency management.

But:

- WSL permissions
- mounted filesystem ownership
- git submodules

can still create issues.

---

4. OpenROAD Setup Takes Time

This is normal.

Even experienced engineers spend time debugging:

- CMake
- Boost

- dependency versions
- package discovery

---

5. Toolchain Setup Is Harder Than RTL Initially

For beginners:

- Verilog is easier
- environment management is harder

This is completely normal.

Recommended Final Workflow

Step 1

Create clean project directory.

Step 2

Clone recursively.

Step 3

Install all dependencies first.

Step 4

Verify SWIG version.

Step 5

Build using moderate thread count.

Recommended:

```
-j4
```

Step 6

Never move build directories.

Step 7

If dependency issues appear:

- delete build cache
- rerun CMake
- verify package versions

Recommended Verification Checklist

Before starting RTL-to-GDS flow verify:

```
which openroad
which yosys
which klayout
which swig
```

Then:

```
openroad -version
yosys -V
```

Typical Beginner Mistakes

Mistake	Result
Not cloning recursively	Missing submodules

Mistake	Result
Using old SWIG	Configuration failure
Too many threads	Compiler killed
Moving build directories	CMake path mismatch
Mixing Boost versions	CMake dependency conflicts
Ignoring build logs	Hard debugging

Final Outcome

After resolving all issues successfully:

- OpenROAD installed
- Yosys installed
- KLayout working
- Full RTL-to-GDSII environment functional
- UART project ready for synthesis and physical design

Final Advice

Do not expect OpenROAD setup to work perfectly on first attempt.

The installation process itself teaches:

- Linux debugging
- dependency management
- CMake behavior
- ASIC toolchain structure
- build systems
- environment handling

These are valuable engineering skills themselves.

Once the environment works:

future RTL-to-GDS projects become dramatically easier.

The first successful setup is the hardest one.